

QNX - ARCHITECTURE

Processes

Process 1:

- **Network Ingestion:** Listens for 100 concurrent patient streams over TCP (Ports 5000+) for discrete vitals and UDP (Ports 6000+) for high-frequency ECG waveforms.
- **Thread Pool:** Utilizes 10 `PatientTCPPoolWorker` threads, where each thread handles exactly 10 non-blocking TCP sockets via `poll()` multiplexing.
- **Data Parsing:** Extracts raw, real-time physiological parameters (Temperature, MAP, HR, SpO2, RR, BP, etc.) from JSON packets using the `cJSON` library

Process 2:

- **Dynamic Load-Shedding:** Calculates a Real-Time Clinical Indicator (RTCI) using $NEWS2 + (APACHE\ II * 0.15)$. This score is injected into the QNX Pulse as a priority value ($severity * 100$), forcing the display scheduler to preempt standard rendering and prioritize critical UI alerts.
- **Hardware Rendering:** Bypasses standard UI toolkits to write pixels directly to the display hardware via the QNX Screen API (`screen/screen.h`), ensuring high-FPS ECG plotting.
- **External AI Integration:** Spawns asynchronous worker threads to fetch detailed patient history and generative AI

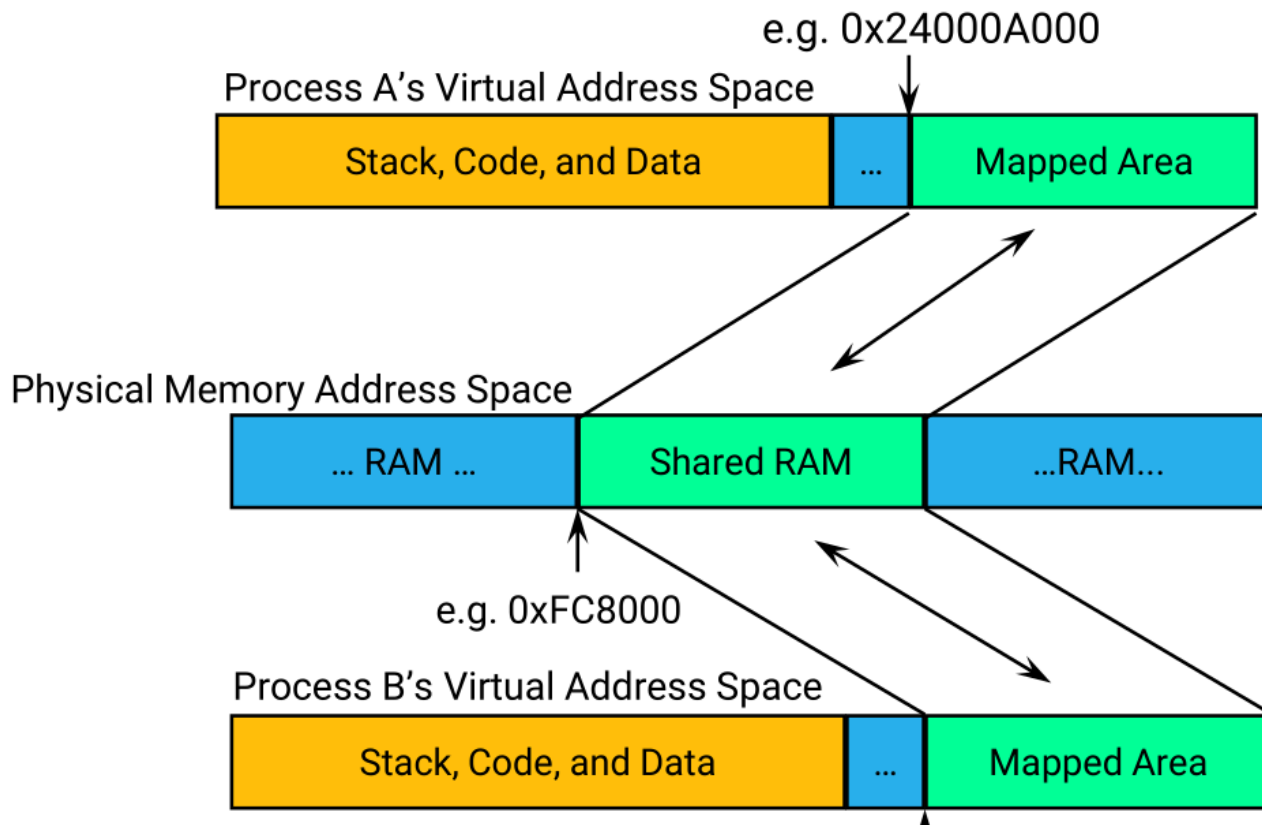
clinical summaries from an external secondary server (172.17.155.171:5000) when a clinician interacts with the display.

Inter Process Communication (IPC):

- **Bulk Data Transfer (Shared Memory):** Process 1 writes parsed vitals and 400-point ECG arrays directly to a POSIX shared memory object (`/patient_data_v2`). Process 2 reads this memory mapped region (`g_shm`) locklessly for screen rendering.
- **Asynchronous Alerts (QNX Pulses):** When Process 1 detects a `"critical": true` flag in the JSON stream, it immediately calculates the patient severity and fires a QNX Pulse to the `CriticalDisplayService` channel attached to Process 2.

Functions Used

Shared Memory



```
fd = shm_open( "/myname", O_RDWR|O_CREAT, 0600 );
```

- name should start with leading / and contain only one /
- using `O_EXCL` can help do synchronization for the case where you have multiple possible creators

```
ftruncate( fd, SHARED_SIZE );
```

- this allocates `SHARED_SIZE` bytes of RAM associated with the shared memory object
 - this will be rounded up to a multiple of the page size, 4K

```
ptr = mmap( NULL, SHARED_SIZE, PROT_READ|PROT_WRITE,
            MAP_SHARED, fd, 0 );
```

- this returns a pointer to (i.e. virtual address for) the shared memory object
- the next step would be to initialize the internal data structures of the object

```
close( fd );
```

- you no longer need the `fd`, so you can close it
-

Shared Memory - Access

To access a shared memory object:

```
fd = shm_open( "/myname", O_RDWR, 0 );
```

- same name that was used for the creation

```
ptr = mmap( NULL, SHARED_SIZE, PROT_READ|PROT_WRITE,  
            MAP_SHARED, fd, 0 );
```

- for read-only access (view), don't use **PROT_WRITE**
- you can gain access to sub-sections of the shared memory by specifying an offset instead of 0, and a different size
 - mapping will be on page-size boundaries, even if offset and size aren't

```
close(fd);
```

- you no longer need the fd, so you can close it

Shared Memory - Cleanup

The allocated memory will be freed when there are no further references to it:

- each fd, mapping, and the name is a reference
- can explicitly close, and unmap:

```
close(fd);  
munmap( ptr, SHARED_SIZE );
```

- on process death, all fds are automatically closed and all mapping unmapped

- the name must be explicitly removed:

```
shm_unlink( "/myname" );
```

- during development and testing this can be done from the command line:

```
rm /dev/shmem/myname
```

Shared Memory Handles

The shared memory handle functions:

- handles are single-use opaque objects used for coordinating access to a shared memory object between two processes

```
shm_create_handle( fd, pid, perms, &handle, flags );
```

- create a **handle** that will allow process **pid** to access the shared memory object **fd**

```
fd = shm_open_handle( handle, perms );
```

```
fd = shm_open_handle_pid( handle, perms, pid );
```

- convert the **handle** to an **fd** that can be used to map the shared memory object
- the **pid** version allows a bit more security by specifying the process creating the shared memory

© QNX a division of BlackBerry 2025 123

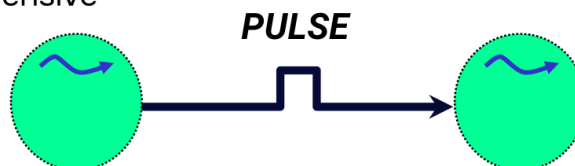
Pulses

QNX Inter-Process Communication

Pulses

Pulses:

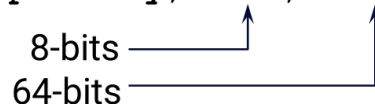
- non-blocking for the sender
- fixed-size payload
 - 64 bit value
 - can be treated as pointer, long, or int
 - 8 bit code (-128 to 127)
 - negative values reserved for system use
 - 7-bits available
- unidirectional (no reply)
- fast and inexpensive



Sending Pulses

Pulses are sent as follows:

```
MsgSendPulse (coid, priority, code, value);
```



- **code** is usually used to mean "pulse type"
 - valid range is `_PULSE_CODE_MINAVAIL` (0) to `_PULSE_CODE_MAXAVAIL` (127)
- to send a pointer use `MsgSendPulsePtr()` instead
- **priority** indicates the priority the receiving thread should run at
 - QNX uses a priority inheritance scheme to minimize priority inversion problems
 - delivery order is based on priority
 - -1 can be used for the priority of the calling thread
- to send a pulse across process boundaries, the sender must have appropriate privilege

Receiving Pulses

Pulses are received just like messages, with a `MsgReceive*` call:

- a server can determine whether it has received a pulse vs. a message by the return value from `MsgReceive()`
 - the return value will be `>0` if a **message** was received.
 - this value will be the **rcvid**, which will be needed to `MsgReply()`
 - the return value will be `== 0` if a **pulse** was received
 - you can not `MsgReply()` to pulses
- the pulse data will be written to the receive buffer
 - the receive buffer must be large enough to hold the pulse structure, if not:
 - the return value will be `== -1`, and **errno** will be **EFAULT**
 - the pulse will be lost

Multi threading

Functions for setting attributes

pthread_attr_init(), pthread_attr_destroy()

pthread_attr_setdetachstate(), *pthread_attr_setinheritsched()*,
pthread_attr_setschedparam(), *pthread_attr_setschedpolicy()*,
pthread_attr_setstacksize(), ...

Setting priority and scheduling algorithm:

- scheduling policies available include:

- **SCHED_FIFO** – first-in-first-out
- **SCHED_RR** – round-robin
- **SCHED_SPORADIC** – sporadic scheduler
- **SCHED_OTHER** – same as round-robin
- **SCHED_NOCHANGE** – keep same policy, but change parameters (e.g. priority)

Thread Attributes - Stack Allocation

You can control the thread's stack size:

– to set the stack size:

```
pthread_attr_setstacksize (&attr, size);
```

- this must be at least **PTHREAD_STACK_MIN**
- this will be rounded up to a multiple of the page-size (4K)
- a guard page will be allocated as well, but no RAM will be used for it
 - this can be controlled with *pthread_attr_setguardsize()*
- the default stack size for new threads is 256K (plus a 4K guard page)
 - the “main” thread gets a 512K stack by default

APACHEE II Score Calculation

A

APACHE II scoring system

[A] Total acute physiology score (APS)

	4	3	2	1	0	1	2	3	4
Body Temp.(°C)	≤29.9	30~31.9	32~33.9	34~35.9	36~38.4	38.5~38.9		39~40.9	≥41
Mean BP (mmHg)	≤49		50~69		70~109		110~129	130~159	≥160
Pulse (/min)	≤39	40~54	55~69		70~109		110~139	140~179	≥180
Respiratory Rate (/min)	≤5		6~9	10~11	12~24	25~34		35~49	≥50
A-a DO2(FIO2 ≥ 0.5)					<200		200~349	350~499	≥500
PaO2(FIO2 < 0.5)	<55	55~60		61~70	>70				
Arterial blood pH	<7.15	7.15~7.24	7.25~7.32		7.33~7.49	7.50~7.59		7.60~7.69	≥7.70
No ABG data; HCO3 ⁻	<15	15~17.9	18~21.9		22~31.9	32~40.9		41~51.9	≥52
Serum sodium(mmol/L)	≤110	111~119	120~129		130~149	150~154	155~159	160~179	≥180
Serum Potassium(mmol/L)	<2.5		2.5~2.9	3.0~3.4	3.5~5.4	5.5~5.9		6.0~6.9	≥7.0
Serum Creatinine (mg/dL)			<0.6		0.6~1.4		1.5~1.9	2.0~3.4	≥3.5
Hematocrit (%)	<20		20~29.9		30~45.9	46~49.9	50~59.9		≥60
WBC(× 10 ³ /mm ³)	<1		1~2.9		3~14.9	15~19.9	20~39.9		≥40
Glasgow coma scale	15 — Glasgow coma scale								

[B] Age points

Age	Score
≤44	0
45~54	2
55~64	3
65~74	5
≥75	6

[C] Chronic Health Points(CHP)

Chronic organ insufficiency	Score
And non operative	5
And emergent postoperative	5
And elective postoperative	2

APACHEII score = [A] APS + [B] Age points+ [C] CHP

B

Modified APACHE II scoring system

[A] Total acute physiology score (APS)

	4	3	2	1	0	1	2	3	4
Mean BP (mmHg)	≤49		50~69		70~109		110~129	130~159	≥160
Serum Sodium (mmol/L)	≤110	111~119	120~129		130~149	150~154	155~159	160~179	≥180
Serum Potassium (mmol/L)	<2.5		2.5~2.9	3.0~3.4	3.5~5.4	5.5~5.9		6.0~6.9	≥7.0
Serum Creatinine (mg/dL)			<0.6		0.6~1.4		1.5~1.9	2.0~3.4	≥3.5
Glasgow coma scale	15 — Glasgow coma scale								

[B] Age points

Age	Score
≤44	0
45~54	2
55~64	3
65~74	5
≥75	6

Modified APACHEII score

= [A] Mean BP, Sodium, Potassium, Creatinine and GCS + [B] Age points

C

APACHE-HF scoring system

	0	1		Cut off	AUC	p value
Mean BP (mmHg)	91 ~	~ 90	Mean BP (mmHg)	91.5	0.678	< 0.001
Pulse (/min)	111 ~	~ 110	Pulse (/min)	110.5	0.594	0.008
Serum sodium(mmol/L)	138 ~	~ 137	Serum sodium(mmol/L)	137.5	0.613	0.001
Serum Potassium(mmol/L)	~4.8	4.9 ~	Serum Potassium(mmol/L)	4.85	0.601	0.004
Serum Creatinine (mg/dL)	~ 1.47	1.48 ~	Serum Creatinine (mg/dL)	1.475	0.676	< 0.001
Hematocrit (%)	37.0 ~	~ 36.9	Hematocrit (%)	36.95	0.617	0.001
Glasgow coma scale	14 ~	~ 13	Glasgow coma scale	13.5	0.567	0.058
Age	~ 71	72 ~	Age	71.5	0.572	0.042

APACHE- HF score = sum total score of each factor as shown above right table

Chart 1: The NEWS scoring system

Physiological parameter	Score						
	3	2	1	0	1	2	3
Respiration rate (per minute)	≤8		9–11	12–20		21–24	≥25
SpO ₂ Scale 1 (%)	≤91	92–93	94–95	≥96			
SpO ₂ Scale 2 (%)	≤83	84–85	86–87	88–92 ≥93 on air	93–94 on oxygen	95–96 on oxygen	≥97 on oxygen
Air or oxygen?		Oxygen		Air			
Systolic blood pressure (mmHg)	≤90	91–100	101–110	111–219			≥220
Pulse (per minute)	≤40		41–50	51–90	91–110	111–130	≥131
Consciousness				Alert			CVPU
Temperature (°C)	≤35.0		35.1–36.0	36.1–38.0	38.1–39.0	≥39.1	